KTÜ PRESS

2026–2027

KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

WEEK 3

Search I: Systematic Search

Constraint Programming

Learning phases: 1 · Pre-instruction 2 · Content 3 · Participation 4 · Assessment 5 · Follow-through

Assoc. Prof. Tolga Berber
KTU Faculty of Science, Department of Computer Science · https://avesis.ktu.edu.tr/tberber

CP

1 · Pre-instruction

Why It Matters?

Week 1 treated CP-SAT as a solver that “just returns a solution”; Week 2 opened the box halfway with propagation.

- Placing one queen per row on an 8×8 board already has 8^8 – over 16 million – raw ways to do it, before you even start checking for attacks
- Trying every one, then checking, is hopeless past a small board
- Backtracking never builds most of those boards at all: the moment a partial placement is already broken, it stops right there

This week opens the box the rest of the way – the search loop that runs underneath every CP-SAT call.

This Week

◎ Learning Objectives

By the end of this week, you will be able to:

1. Explain what backtracking search is and trace a search tree by hand for a small CSP, and explain why it is complete
2. Compare variable-ordering heuristics (MRV, degree, dom/wdeg) and the value-ordering heuristic LCV, and select an ordering by hand
3. Explain forward checking and MAC as two look-ahead techniques of different strength, and trace their effect on domains by hand
4. Configure a CP-SAT model’s decision strategy, worker count and time limit, and read the solver’s search statistics
5. Choose and implement, in CP-SAT, an ordering-plus-look-ahead configuration suited to a hard instance, and measure the resulting difference in solving effort

UNIT 1

Backtracking and the Search Tree

2 · Content Backtracking: Depth-First Over Partial Assignments

- Week 1's N-queens paper model (`Q1..Q4`, one variable per row) and Week 2's AC-3 propagation fixpoint are both still in play
- At each step: choose the next unassigned variable (this unit uses a fixed order – Unit 2 revisits the choice itself)
- Try each value in its domain in turn; check the partial assignment against every constraint touching only assigned variables; recurse if consistent

Definition: Backtracking search
Chronological depth-first search over partial assignments: when the current assignment cannot be extended with any value of the current variable, undo the most recent choice and try the previous choice point's next alternative.

The Search Tree: Root, Nodes, Edges, Leaves

- **Root** = the empty assignment
- **Internal node** = a partial assignment together with the next variable to branch on
- **Edge** out of a node = one candidate value for that variable
- **Leaf** = either a solution (complete, consistent) or the endpoint of a pruned branch

Definition: Dead end (failure node)
A node at which every value in the current variable's domain has been tried and rejected – forces the search to *backtrack*: undo the most recent choice and try the previous variable's next untried value.

Analogy: Filling a Sudoku Grid in Pencil

Analogy

You write a candidate digit, and the moment it conflicts with a digit already in the same row, column or box, you erase just that one digit and try the next candidate – you never erase the whole grid and start over.

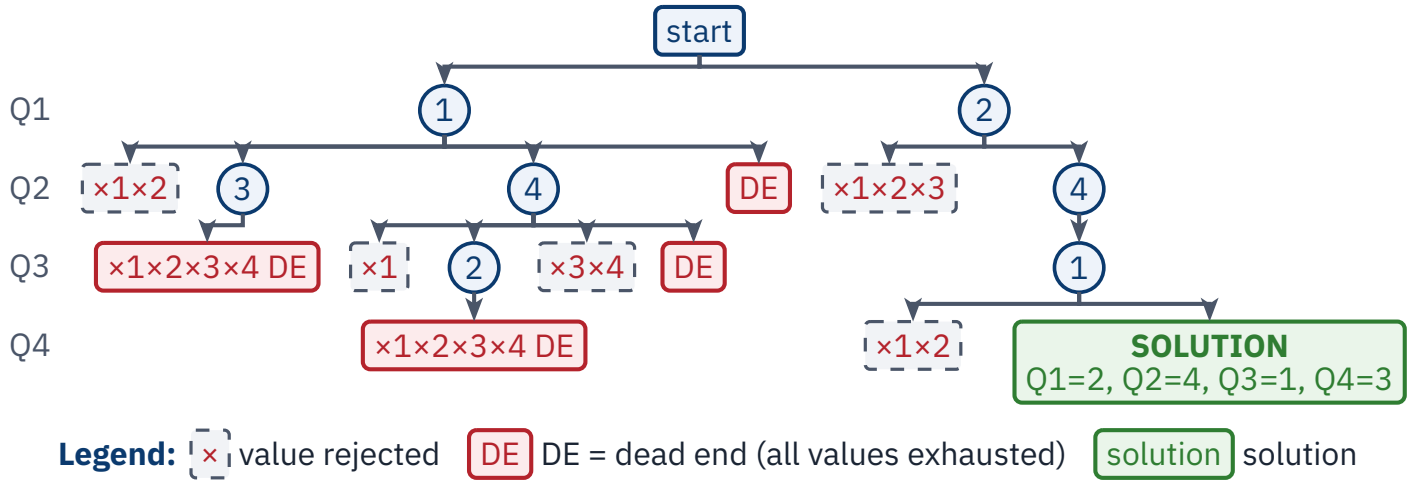
Backtracking search does exactly this at every level of the tree: undo one choice, not the whole branch.

Worked by Hand: 4-Queens, `Q1,Q2,Q3,Q4`, Values `1..4`

Variable order `Q1,Q2,Q3,Q4`; value order ascending `1,2,3,4`.

- `Q1=1`: three dead ends in a row – every branch under `Q2=3` and `Q2=4` fails, so `Q1=1` itself becomes exhausted
- Backtrack to the root: `Q1=2`, `Q2=4`, `Q3=1`, `Q4=3` – consistent with every earlier variable
- Solution found: `Q1=2, Q2=4, Q3=1, Q4=3`

The Full Trace, Drawn as a Tree



Three dead ends, all under `Q1=1`; the `Q1=2` branch succeeds on its first fully-explored path.

Completeness: Why the Search Never Gives Up Too Early

Definition: Completeness

Because backtracking systematically tries every value at every level, skipping only branches already proven inconsistent, it is guaranteed to eventually find a solution or correctly prove none exists – infeasibility is declared only once the *root itself* becomes a dead end.

Contrast: Week 4’s local search gives up this guarantee in exchange for speed on much larger instances.

3 · Participation Quick Check: After a Dead End

In the 4-queens trace, after `Q1=1`, `Q2=3`, every value tried for `Q3` fails. Name the operation the search performs next, and explain why it does not simply declare the whole problem infeasible.

The operation is *backtrack* – return to the previous choice point (`Q2`) and try its next untried value. Infeasibility is only declared once every value at every level has been exhausted, i.e. once the root itself becomes a dead end.

UNIT 2

Variable and Value Ordering Heuristics

2 · Content Variable Ordering: Minimum Remaining Values

- Unit 1’s three dead ends all happened under `Q1=1` – an entire sub-tree wasted before the working branch was ever reached
- Variable-ordering heuristics replace Unit 1’s fixed order with a rule for *which* unassigned variable to branch on next

Definition: Minimum Remaining Values (MRV)
Always branch on the unassigned variable with the fewest legal values left in its current domain – “fail-first”: if a branch is doomed, better to discover it high in the tree, before wasted work accumulates beneath it.

Degree Heuristic and dom/wdeg

Definition: Degree heuristic
Pick the unassigned variable involved in the most constraints with other unassigned variables – commonly used to break MRV ties.

- **dom/wdeg**: divides a variable's current domain size by a *weight* accumulated on each of its constraints every time that constraint has caused a search failure so far
- Unlike MRV/degree, it *learns from the search's own history* – constraints that caused trouble before get picked earlier next time

Value Ordering: Least Constraining Value

Definition: Least Constraining Value (LCV)
Once a variable is chosen, try first the value that rules out the fewest remaining choices for neighbouring unassigned variables – “succeed-first”, the deliberate opposite intention from MRV.

- MRV wants to fail fast; LCV wants to succeed fast – two different instincts, applied at two different decision points
- Not competitors: choose the variable by MRV, then order that variable’s values by LCV – the two compose directly

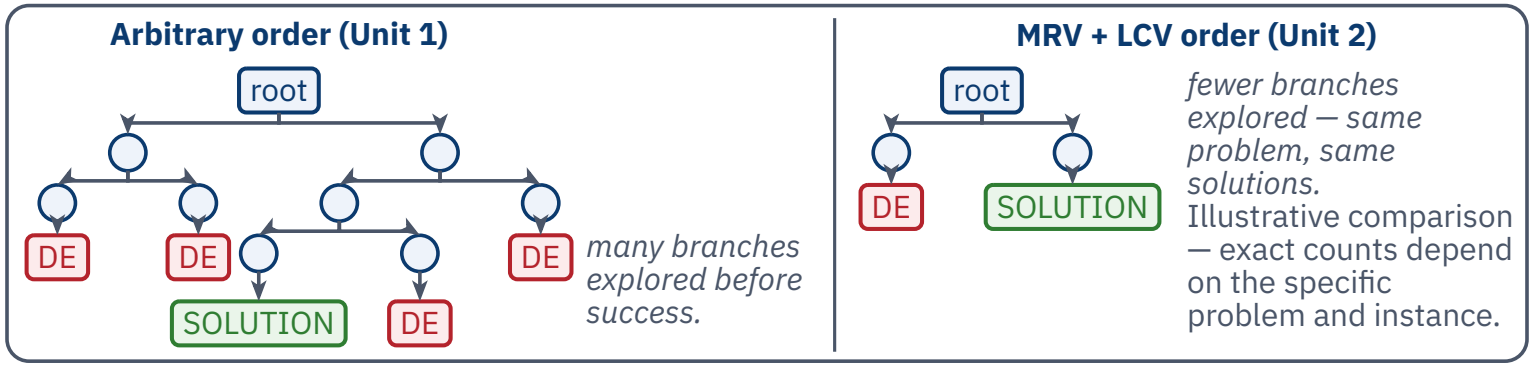
Analogy: Fail-First and Succeed-First

Analogy

MRV is like scheduling the exam with the fewest open rooms or time slots first: if a clash is coming, you want to discover it while placing the most constrained exam, not after nine easy ones are already locked in. LCV is like choosing the travel connection that leaves the most onward options open for the rest of your itinerary, rather than the one that happens to be cheapest right now but locks you into a single remaining route.

Same Problem, Same Solutions, Different Explored Tree

Variable-ordering and value-ordering heuristics are not competitors – the search tree's *shape* stays the problem's shape; only the *order* in which nodes are visited changes.



Illustrative comparison: an arbitrary order explores a bushy tree; MRV+LCV explores a narrower one – same problem, same solutions, exact counts depend on the specific instance.

3 · Participation Quick Check: Which Variable Does MRV Select?

Three unassigned variables remain: `A` (domain size 3, touches 2 others), `B` (domain size 1, touches 3 others), `C` (domain size 2, touches 1 other). Which variable does MRV select next?

MRV picks the smallest current domain – `B` (size 1) – regardless of its degree; no tie exists here, so the degree heuristic is not needed as a tie-break in this particular case.

UNIT 3

Forward Checking and MAC

2 · Content The Gap This Unit Closes

- Plain backtracking (Unit 1) is *look-back only* – it checks constraints against already-assigned variables and only discovers a failure once every value of the *current* variable has been tried
- Unit 2 changed the *order* of the search tree; this unit changes *how much of the tree exists in the first place*, by pruning other variables' domains before a doomed branch is tried
- Both techniques reuse Week 2's AC-3 – the only change is *when* it runs: once before search began, now after every single assignment during search

UNIT 3 · FC VS MAC

Forward Checking: One Hop Outward

Definition: Forward checking
The instant a variable is assigned, remove every now-inconsistent value from the domain of each *directly* constrained still-unassigned variable; if any of those domains becomes empty, backtrack immediately.

A one-hop check outward from the just-assigned variable – it does not propagate further through the rest of the constraint graph.

UNIT 3 · FC VS MAC

MAC: Maintaining Arc Consistency

Definition: Maintaining Arc Consistency (MAC)
After every assignment, re-run full arc consistency (Week 2's AC-3, unchanged) over all remaining unassigned variables, not just the one-hop neighbours – strictly more pruning, at the cost of more work per node.

The classic "cheap and shallow" versus "expensive and thorough" look-ahead trade-off.

UNIT 3 · FC VS MAC

Worked by Hand: A, B, C on a Path

A != B, B != C, no direct A–C constraint; domains A:{1,2}, B:{1,2}, C:{1,2}. Assign A=1.

- Forward checking removes 1 from B (B:{2}), but leaves C untouched (C:{1,2}) – C is not a direct neighbour of A
- MAC continues propagating: since B is now forced to 2, MAC also removes 2 from C (C:{1}) – a narrowing forward checking misses here

Before / Forward Checking / MAC

Forward checking versus MAC: domain updates after $A = 1$

Before assignment	
Var	**Domain**
A	{1, 2}
B	{1, 2}
C	{1, 2}

After forward checking ($A = 1$)	
Var	**Domain**
A	{**1**} *(assigned)*
B	{~~1~~, **2**}
C	{1, 2} *(untouched)*

After MAC ($A = 1$)	
Var	**Domain**
A	{**1**} *(assigned)*
B	{~~1~~, **2**}
C	{~~2~~, **1**}



Forward checking only updates *A*'s direct neighbour *B*, leaving *C*'s domain untouched (amber flag); MAC continues propagating the singleton domain of *B* and also narrows *C*, which forward checking never reaches.

Forward checking only updates `A`'s direct neighbour `B`; MAC continues propagating and also narrows `C`.

UNIT 3 · FC VS MAC

Analogy: One Booking Versus a Full Rebooking Check

Analogy

Forward checking is like crossing a booked room off every other still-unscheduled exam's shortlist the moment you book it. MAC is like a department-wide rebooking system that, after every new booking, re-checks and updates every other still-open booking's shortlist, not just the one directly affected – more thorough, at the cost of rechecking more of the schedule each time.

3 · Participation

Quick Check: Domains After Forward Checking

Using the `A, B, C` example (domains `{1,2}` each), compute `B`'s and `C`'s domains immediately after forward checking runs on `A=1`.

Forward checking removes `1` from `B` (`B != A`), giving `B:{2}`; `C` is not directly constrained with `A`, so it stays unchanged: `C:{1,2}` – exactly the gap MAC closes.

UNIT 4

Search Parameters in CP-SAT

2 · Content Decision Strategies in CP-SAT

- `model.AddDecisionStrategy(variables, var_strategy, value_strategy)` fixes which variable- and value-selection rule CP-SAT's search uses over a given list of variables
- `CHOOSE_FIRST` (Unit 1's fixed order), `CHOOSE_MIN_DOMAIN_SIZE` (CP-SAT's MRV analogue), `SELECT_MIN_VALUE`/`SELECT_MAX_VALUE` (Unit 2's LCV intuition, as concrete choices)
- CP-SAT has no literal "degree" or "dom/wdeg" enum value
- Its *default* automatic search already blends several internal heuristics and usually outperforms a hand-picked one

UNIT 4 · CP-SAT SEARCH

Code: The Petersen Graph – 10 Nodes, 15 Edges

Larger and structurally harder than Week 1's 7-node Australia toy, so ordering and worker choices make a visible difference.

```
colours = [model.NewIntVar(0, 2, f"colour_{node}") for node in range(NUM_NODES)]
return colours
```

```
for node_a, node_b in EDGES:
    model.Add(colours[node_a] != colours[node_b])
```

Code and Output: Default Search on Petersen

```
39 solver = cp_model.CpSolver()
40 solver.parameters.num_search_workers = 1
41 solver.parameters.random_seed = 42
42 status = solver.Solve(model)
43 print_search_stats(solver, status)
```

Expected Output

```
Status: OPTIMAL
NumBranches: 69
NumConflicts: 1
WallTime: 0.0009
```

Code and Output: Forced MRV + Min-Value Ordering

```
54 model.AddDecisionStrategy(
55     colours, cp_model.CHOOSE_MIN_DOMAIN_SIZE, cp_model.SELECT_MIN_VALUE
56 )
57 solver = cp_model.CpSolver()
58 solver.parameters.search_branching = cp_model.FIXED_SEARCH
59 solver.parameters.num_search_workers = 1
60 solver.parameters.random_seed = 42
61 status = solver.Solve(model)
62 print_search_stats(solver, status)
```

Expected Output

```
Status: OPTIMAL
NumBranches: 65
NumConflicts: 0
WallTime: 0.0008
```

A small drop – 69 to 65 branches – on a graph this size.

UNIT 4 · CP-SAT SEARCH

`num_search_workers` and `max_time_in_seconds`

- `solver.parameters.num_search_workers` – CP-SAT's built-in parallel portfolio; *why* different workers help is Week 4's job, this unit stops at "here is the knob and what it visibly does"
- `solver.parameters.max_time_in_seconds` – a hard wall-clock cutoff, read together with `StatusName`, `NumBranches`, `NumConflicts`, `WallTime`

Code and Output: Parallel Search Workers

```
73 solver = cp_model.CpSolver()
74 solver.parameters.num_search_workers = num_workers
75 solver.parameters.random_seed = 42
76 status = solver.Solve(model)
77 print_search_stats(solver, status)
```

Expected Output

```
num_search_workers=1: NumBranches 69, WallTime 0.0008
num_search_workers=8: NumBranches 69, WallTime 0.0024
```

UNIT 4 · CP-SAT SEARCH

Code: Langford Pairs, n=7

Arrange `1..7`, each value twice, into a length-14 sequence so that value `k`'s two occurrences have exactly `k` entries between them. A solution exists only when $n \equiv 0$ or $n \equiv 3 \pmod 4$; $n = 7$ qualifies.

```
positions = [
    [model.NewIntVar(0, NUM_POSITIONS - 1, f"pos_{k}_{i}") for i in range(2)]
    for k in range(1, N + 1)
]
return positions
```

```
model.AddAllDifferent([entry for pair in positions for entry in pair])
for k in range(1, N + 1):
    first, second = positions[k - 1]
    model.Add(second - first == k + 1)
```

Code and Output: Default Search on Langford(7)

```
55 solver = cp_model.CpSolver()
56 solver.parameters.num_search_workers = 1
57 solver.parameters.random_seed = 42
58 solver.parameters.max_time_in_seconds = 10
59 status = solver.Solve(model)
60 print_search_stats(solver, status)
```

Expected Output

```
Status: OPTIMAL
NumBranches: 564
NumConflicts: 21
WallTime: 0.0020
```

Output: Forced Ordering and a Time-Limited Run, For Real

Forced MRV + min-value ordering:

Expected Output

```
Status: OPTIMAL
NumBranches: 843
NumConflicts: 0
WallTime: 0.0025
```

A tightened `max_time_in_seconds`, cut short:

Expected Output

```
Status: UNKNOWN
NumBranches: 0
NumConflicts: 0
WallTime: 0.0010
```

UNIT 4 · CP-SAT SEARCH

Code: The Magic Square, For Real

16 cells, domain `1..16`, `AllDifferent` plus 10 equal-sum constraints (4 rows, 4 columns, 2 diagonals), each equal to 34.

```
45 solver = cp_model.CpSolver()
46 solver.parameters.num_search_workers = 1
47 solver.parameters.random_seed = 42
48 status = solver.Solve(model)
49 print_search_stats(solver, status)
50 grid = [[solver.Value(cells[r][c]) for c in range(N)] for r in range(N)]
51 for row in grid:
52     print(row)
```

UNIT 4 · CP-SAT SEARCH

Output: The Magic Square, For Real

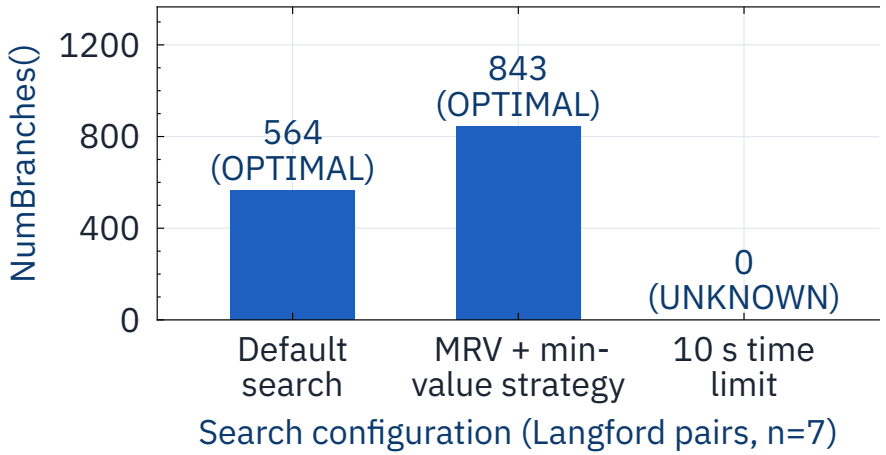
Expected Output

```
Status: OPTIMAL
NumBranches: 3107
NumConflicts: 53
WallTime: 0.0119
[8, 1, 10, 15]
[3, 13, 6, 12]
[14, 4, 11, 5]
[9, 16, 7, 2]
```

The actual completed grid – every row, column and both diagonals sum to 34.

UNIT 4 · CP-SAT SEARCH

Same Model, Different Parameters, Different Effort



Solved to completion (OPTIMAL)

Cut short by time limit (UNKNOWN)

Real CP-SAT statistics from langford7-default-stats.txt, langford7-ordered-stats.txt, langford7-time-limited-stats.txt — mixed result, no configuration highlighted as a winner.

Langford(7) under default search, a forced decision strategy and a tightened time limit – real captured numbers, no invented values.

3 · Participation Quick Check: Compare `NumBranches`

Run Langford(7) once with CP-SAT's default search and once forcing `CHOOSE_MIN_DOMAIN_SIZE` + `SELECT_MIN_VALUE` (both capped at `max_time_in_seconds=10`). Which run's `NumBranches()` is smaller?

The default run's `NumBranches` (564) is smaller than the forced-ordering run's (843) in this captured run – a reminder that forcing a decision strategy is not automatically faster than CP-SAT's own default search.

Summary

SUMMARY

Summary

Summary

- Backtracking search explores partial assignments depth-first; a dead end forces backtrack to the previous choice point, and completeness follows because every value at every level is eventually tried
- Variable-ordering (MRV, degree, dom/wdeg) and value-ordering (LCV) heuristics change the *order* the same tree is explored, not its underlying shape
- Forward checking prunes one hop outward after each assignment; MAC re-establishes full arc consistency every time – more pruning, more work per node
- CP-SAT exposes this whole loop through `AddDecisionStrategy`, `num_search_workers` and `max_time_in_seconds`, readable through its own search statistics

SUMMARY

4 · Assessment

Check Yourself

- One Check Yourself question closes each unit in this week's chapter – answers are collected at the chapter's end
- Why look-back-only backtracking is still complete, an MRV/degree tie-break, why MAC's extra work can pay off, one legitimate reason to force a decision strategy by hand

SUMMARY

5 · Follow-through

Transfer — Where Else?

- Every solver you use for the rest of this course – CP-SAT here, Gurobi's branch-and-bound from Week 7 onward – runs a backtracking-shaped search underneath its more sophisticated machinery
- `dom/wdeg`'s "learn from past failures" idea previews Week 4's nogood learning, which generalizes the same principle
- This course has no running capstone project – the thread is the *same* classic problems (graph coloring, magic square, Langford pairs), revisited with new machinery in later weeks

SUMMARY

In the Lab

This week's lab sheet
You hand-trace a second small CSP's backtracking tree, compare an arbitrary order against MRV+LCV on the Petersen graph by hand and then by running `graph_coloring.py`, extend the `A, B, C` example to MAC by hand, and close by running `magic_square.py` and `langford.py` under at least two configurations each.

- Next week: local search trades completeness for speed – min-conflicts, tabu, simulated annealing, restarts and nogood learning
- Keep `search_stats.py` – Week 4 reuses it unchanged to read its own search statistics